

P2613R1

Add the missing `empty` to `mdspan`

Published Proposal, 2022-06-25

This version:

<https://wg21.link/P2613R1>

Author:

[Yihe Li](#)

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++23

Source:

github.com/Mick235711/wg21-papers/blob/main/P2613R1.bs

Issue Tracking:

[GitHub Mick235711/wg21-papers](#)

Table of Contents

1	Revision History
1.1	R1
1.2	R0
2	Motivation
3	Design
4	Wording
4.1	24.7.  Class template <code>mdspan</code> [mdspan.mdspan]
4.1.1	24.7.  .1 Overview [mdspan.mdspan.overview]

References

Normative References

This paper propose to fix a defect in [P0009R17]. During its LWG review, I found that even though the proposed `std::mdspan` type have a `size()` member function, it does not have an `empty()` member function, which makes it distinct from nearly all other STL containers. So this paper propose to add the missing member to increase consistency and performance of common operations.

§ 1. Revision History

§ 1.1. R1

Fixed wording given LWG feedback.

§ 1.2. R0

Initial revision.

§ 2. Motivation

Consistency among library types is important and makes user interaction intuitive. Also, adding `empty()` does have performance optimization chance in common implementations. If we survey the current STL containers, we can summarize as a table for their `empty()` and `size()` behavior:

Having <code>size()</code> without <code>empty()</code>	Having both <code>size()</code> and <code>empty()</code>	Having <code>empty()</code> without <code>size()</code>
<code>std::bitset</code> <code>std::initializer_list</code>	<code>std::vector</code> <code>std::list</code> <code>std::deque</code> <code>std::span</code> ...	<code>std::forward_list</code>

We can see a clear trend in the STL to provide `empty()` with `size()` concurrently whenever possible. The only containers in STL that has `size()` but not `empty()` are some of the fixed-sized containers like `std::bitset`, for which `empty()` does not makes sense. Conversely, even though

`std::forward_list` cannot provide `size()` due to not storing this information, it still provides `empty()` for convenience. More importantly, `std::span` provide an `empty()` too, and since `mdspan` is modeled as a higher-dimensional `std::span`, it should have an `empty()` alongside its `size()` too.

Apart from the consistency motivation, the introduction of `empty()` can have real performance gains to users, since under the as-if wording it is just specified as "the same behaviour as `size() == 0`", but a good implementation can avoid calling `size()` entirely. For example, since `size()` is specified (through `std::extents::fwd-prod-of-extents`) to perform a multiplication on all the extents, `empty()` can simply be implemented as checking whether all the extents are greater than zero, which can be substantially faster than multiplication. This possibility of better performance is also why we were told to rely on `empty()` instead of doing `size() == 0`. Not including `empty()` will both create surprise for users, and also force them to resort to a worse approach to check emptiness.

§ 3. Design

The design of the member function follows that of `std::span`. Which means that it is marked `const`, `noexcept`, and `[[nodiscard]]`.

In theory, since this is a pure addition, it can be done after C++23 ships. However, I would argue (and the `mdspan` authors also agree) that not having both `empty()` and `size()` in [P0009R17] is simply a design mistake, and delay the addition will cause significant user confusion.

§ 4. Wording

The wording below is based on [P0009R17].

§ 4.1. 24.7. Class template `mdspan` [`mdspan.mdspan`]

§ 4.1.1. 24.7. .1 Overview [`mdspan.mdspan.overview`]

```
namespace std {  
  
    template<class ElementType, class Extents, class LayoutPolicy, class AccessorPolicy>  
    class mdspan {  
    public:
```

```

using extents_type = Extents;
using layout_type = LayoutPolicy;
using accessor_type = AccessorPolicy;
using mapping_type = typename layout_type::template mapping<extents_type>
using element_type = ElementType;
using value_type = remove_cv_t<element_type>;
using size_type = typename extents_type::size_type ;
using rank_type = typename extents_type::rank_type ;
using pointer = typename accessor_type::pointer;
using reference = typename accessor_type::reference;

static constexpr rank_type rank() { return extents_type::rank(); }
static constexpr rank_type rank_dynamic() { return extents_type::rank_dyn
static constexpr size_t static_extent(rank_type r) { return extents_type:
constexpr size_type extent(rank_type r) const { return extents().extent(r

// [mdspan.mdspan.ctor], mdspan Constructors
constexpr mdspan();
constexpr mdspan(const mdspan& rhs) = default;
constexpr mdspan(mdspan&& rhs) = default;

template<class... SizeTypes>
    explicit constexpr mdspan(pointer ptr, SizeTypes... exts);
template<class SizeType, size_t N>
    explicit(N != rank_dynamic())
        constexpr mdspan(pointer p, span<SizeType, N> exts);
template<class SizeType, size_t N>
    explicit(N != rank_dynamic())
        constexpr mdspan(pointer p, const array<SizeType, N>& exts);
constexpr mdspan(pointer p, const extents_type& ext);
constexpr mdspan(pointer p, const mapping_type& m);
constexpr mdspan(pointer p, const mapping_type& m, const accessor_type& a

template<class OtherElementType, class OtherExtents,
        class OtherLayoutPolicy, class OtherAccessorPolicy>
    explicit(see below)
        constexpr mdspan(
            const mdspan<OtherElementType, OtherExtents,
                OtherLayoutPolicy, OtherAccessorPolicy>& other);

constexpr mdspan& operator=(const mdspan& rhs) = default;
constexpr mdspan& operator=(mdspan&& rhs) = default;

```

```

// [mdspan.mdspan.members], mdspan members
template<class... SizeTypes>
    constexpr reference operator[](SizeTypes... indices) const;
template<class SizeType>
    constexpr reference operator[](span<SizeType, rank()> indices) const;
template<class SizeType>
    constexpr reference operator[](const array<SizeType, rank()>& indices)

constexpr size_t size() const;
[[nodiscard]] constexpr bool empty() const noexcept;

friend constexpr void swap(mdspan& x, mdspan& y) noexcept;

constexpr const extents_type& extents() const { return map_.extents(); }
constexpr const pointer& data() const { return ptr_; }
constexpr const mapping_type& mapping() const { return map_; }
constexpr const accessor_type& accessor() const { return acc_; }

static constexpr bool is_always_unique() {
    return mapping_type::is_always_unique();
}
static constexpr bool is_always_contiguous() {
    return mapping_type::is_always_contiguous();
}
static constexpr bool is_always_strided() {
    return mapping_type::is_always_strided();
}

constexpr bool is_unique() const {
    return map_.is_unique();
}
constexpr bool is_contiguous() const {
    return map_.is_contiguous();
}
constexpr bool is_strided() const {
    return map_.is_strided();
}
constexpr size_type stride(rank_type r) const {
    return map_.stride(r);
}

private:
    accessor_type acc_; // exposition only

```

```

    mapping_type map_; // exposition only
    pointer ptr_; // exposition only
};

template <class CArray>
requires(is_array_v<CArray> && rank_v<CArray>==1)
mdspan(CArray&)
    -> mdspan<remove_all_extents_t<CArray>, extents<size_t, extent_v<CArray,

template <class Pointer>
requires(!is_array_v<Pointer> && is_pointer_v<Pointer>)
mdspan(Pointer&)
    -> mdspan<remove_pointer_t<Pointer>, extents<size_t>>>

template <class ElementType, class... Integrals>
requires((is_convertible_v<Integrals, size_t> && ...) && sizeof...(Integral
explicit mdspan(ElementType*, Integrals...)
    -> mdspan<ElementType, dextents<size_t, sizeof...(Integrals)>>;

template <class ElementType, class SizeType, size_t N>
mdspan(ElementType*, span<SizeType, N>)
    -> mdspan<ElementType, dextents<size_t, N>>;

template <class ElementType, class SizeType, size_t N>
mdspan(ElementType*, const array<SizeType, N>&)
    -> mdspan<ElementType, dextents<size_t, N>>;

template <class ElementType, class MappingType>
mdspan(ElementType*, const MappingType&)
    -> mdspan<ElementType, typename MappingType::extents_type,
        typename MappingType::layout_type>;

template <class ElementType, class MappingType, class AccessorType>
mdspan(ElementType*, const MappingType&, const AccessorType&)
    -> mdspan<ElementType, typename MappingType::extents_type,
        typename MappingType::layout_type, AccessorType>;

}

```

§ 4.1.2. 24.7.◆.3 Members [mdspan.mdspan.members]

[...]

```
constexpr size_type size() const;
```

Precondition: The size of the multidimensional index space `extents()` is a representable value of type `size_type` ([basic.fundamental]).

Returns: `extents().fwd-prod-of-extents(rank())`.

```
[[nodiscard]] constexpr bool empty() const noexcept;
```

Returns: true if the size of the multidimensional index space `extents()` is 0, otherwise false.

```
friend constexpr void swap(mdspan& x, mdspan& y) noexcept;
```

Effects: Equivalent to:

```
swap(x.ptr_, y.ptr_);  
swap(x.map_, y.map_);  
swap(x.acc_, y.acc_);
```

§ References

§ Normative References

[P0009R17]

Christian Trott; et al. [MDSPAN](https://wg21.link/p0009r17). URL: <https://wg21.link/p0009r17>